Assignment 08: Neural Networks

CS201L: Artificial Intelligence Laboratory
Indian Institute of Technology, Dharwad

05 - Mar - 2026

Problem Statement

You are given a real-world multiclass classification dataset as a CSV file named `activity-detection.csv`, constructed by combining the original training and testing files of the Human Activity Recognition (HAR) using Smartphones Dataset.

The objective is to classify human activities into one of six daily activities based on sensor-based features extracted from smartphone inertial signals.

Dataset Description

The Human Activity Recognition database was built from recordings of 30 participants performing activities of daily living (ADL) while carrying a waist-mounted smartphone with embedded inertial sensors.

Each subject performed six activities:

- WALKING
- WALKING UPSTAIRS
- WALKING DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The smartphone (Samsung Galaxy S II) captured:

- 3-axial linear acceleration (accelerometer)
- 3-axial angular velocity (gyroscope)

at a constant sampling rate of 50Hz. The signals were preprocessed using noise filtering and segmented into fixed-width sliding windows of 2.56 seconds with 50% overlap (128 readings per window).

From each window, a vector of features was extracted from both time and frequency domains.

Dataset Properties

- Total subjects: 30
- Activities: 6 (multiclass)
- Total features: 561 (numerical)
- Feature domains: Time-domain and Frequency-domain
- Sensors: Accelerometer, Gyroscope
- Target attribute: Activity
- Data format: CSV file
- Dataset file: `activity-detection.csv`

Attribute Information

For each record in the dataset, the following information is provided:

- Triaxial acceleration from the accelerometer (total acceleration)
- Triaxial body acceleration (body motion component)
- Triaxial angular velocity from the gyroscope
- Gravity acceleration components
- 561-feature vector extracted from time and frequency domains
- Activity label (multiclass target)
- Subject identifier (ID of participant)

Dataset Statistics

Property	Value
Total Samples	10,299
Total Features	561
Target Classes	6
Missing Values	0
Data Types	float64 (561), int64 (1), object (1)

Table 1: Overall Dataset Information

Activity	Sample Count	Percentage (%)
LAYING	1,944	18.88
STANDING	1,906	18.51
SITTING	1,777	17.25
WALKING	1,722	16.72
WALKING UPSTAIRS	1,544	14.99
WALKING DOWNSTAIRS	1,406	13.65
Total	**10,299**	**100.00**

Table 2: Activity-wise Sample Distribution

Prepared Dataset Details

You are provided with 12 CSV files representing 4 dataset variations, each split into training (60%), validation (20%), and testing (20%):

- Original data: '`Activity_Train.csv`', '`Activity_Validation.csv`', '`Activity_Test.csv`'.
- Standardized data: '`Activity_Scaled_Train.csv`', '`Activity_Scaled_Validation.csv`', '`Activity_Scaled_Test.csv`'.
- PCA transformed dataset (all components): '`Activity_PCAAll_Train.csv`', '`Activity_PCA-All_Validation.csv`', '`Activity_PCAAll_Test.csv`'.
- PCA transformed dataset (99% variance): '`Activity_PCA99_Train.csv`', '`Activity_PCA99_Validation.csv`', '`Activity_PCA99_Test.csv`'.

Note: the '`subject`' *attribute has been removed, since it is an identifier and not a predictive attribute.*

Write a Python program to complete the following tasks.

Part A: Single and Two hidden layer Neural network architecture on the Original Dataset

Perform neural network classification and performance metric evaluation as listed below.

1. Load train, validation, and test data from original data: '`Activity_Train.csv`', '`Activity_Validation.csv`', '`Activity_Test.csv`', respectively.
2. Implement a neural network for classification using PyTorch with the following specifications:
   - Input layer: 561 linear neurons (corresponding to real-valued input features).
   - All neurons in the hidden layers use the tanh activation function.
   - Hidden layers: Experiments with different numbers of hidden layers and different numbers of neurons in each hidden layer:
     (a) Single hidden layer architectures:
       – Architecture 1: Input layer → hidden layer (561 neurons) → Output layer
       – Architecture 2: Input layer → hidden layer (1024 neurons) → Output layer
       – Architecture 3: Input layer → hidden layer (128 neurons) → Output layer
     (b) Two hidden layer architectures:
       – Architecture 1: Input layer → hidden layer (561 neurons) → hidden layer (561 neurons) → Output layer
       – Architecture 2: Input layer → hidden layer (1024 neurons) → hidden layer (128 neurons) → Output layer
       – Architecture 3: Input layer → hidden layer (1024 neurons) → hidden layer (64 neurons) → Output layer
       – Architecture 4: Input layer → hidden layer (256 neurons) → hidden layer (64 neurons) → Output layer
   - Output layer: 6 neurons ($Softmax$ activation function).
3. Train the neural network using:
   - Optimizer: Stochastic Gradient Descent (SGD)
   - Loss function: Cross Entropy

4. After completion of the training save the model to a folder. *Hint: take a look at corresponding sample code*
5. Plot a graph for epoch vs loss for training dataset for all architecture.
6. Evaluate the different architectures using validation data and obtain their accuracy.
7. Choose one architecture from single hidden layer and one from two hidden layers(with best validation accuracy).
8. Evaluate the best chosen architecture from single hidden layer and two hidden layer architectures using test data and print following metrics
   - Confusion matrix
   - Accuracy
   - Micro and macro averages of precision, recall, and F1-score.

Part B: Single and Two hidden layer Neural network architecture on the Standardized Dataset

Perform neural network classification and performance metric evaluation as listed below.

1. Load train, validation, and test data from standardized data: '`Activity_Scaled_Train.csv`', '`Activity_Scaled_Validation.csv`', '`Activity_Scaled_Test.csv`', respectively.
2. Implement a neural network for classification using PyTorch with the following specifications:
   - Input layer: 561 linear neurons (corresponding to real-valued input features).
   - All neurons in the hidden layers use the tanh activation function.
   - Hidden layers: Experiment with different numbers of hidden layers and different numbers of neurons in each hidden layer:
     (a) Single hidden layer architectures:
       – Architecture 1: Input layer → 10 neurons → Output layer
       – Architecture 2: Input layer → 100 neurons → Output layer
       – Architecture 3: Input layer → hidden layer (64 neurons) → Output layer
       – Architecture 4: Input layer → hidden layer (512 neurons) → Output layer
     (b) Two hidden layer architectures:

  - Architecture 1: Input layer → hidden layer (64 neurons) → 16 neurons → Output layer
  - Architecture 2: Input layer → hidden layer (128 neurons) → hidden layer (32 neurons) → Output layer
  - Architecture 3: Input layer → hidden layer (256 neurons) → hidden layer (64 neurons) → Output layer
  - Architecture 4: Input layer → hidden layer (512 neurons) → hidden layer (128 neurons) → Output layer
- Output layer: 6 neurons ($Softmax$ activation function).

3. Train the neural network using:
   - Optimizer: Stochastic Gradient Descent (SGD)
   - Loss function: Cross Entropy
4. After completion of the training save the model to a folder. *Hint: take a look at corresponding sample code*
5. Plot a graph for epoch vs loss for training dataset for all architecture.
6. Evaluate the different architectures using validation data and obtain their accuracy.
7. Choose one architecture from single hidden layer and one from two hidden layers(with best validation accuracy).
8. Evaluate the best chosen architecture from single hidden layer and two hidden layer architectures using test data and print following metrics
   - Confusion matrix
   - Accuracy
   - Micro and macro averages of precision, recall, and F1-score.

Part C: Single and Two hidden layer Neural network architecture on the PCA Transformed Dataset (All Components)

Perform neural network classification and performance metric evaluation as listed below.

1. Load train, validation, and test data from PCA transformed dataset (all components): '`Activity_PCAAll_Train.csv`', '`Activity_PCAAll_Validation.csv`', '`Activity_PCAAll_Test.csv`', respectively.
2. Implement a neural network for classification using PyTorch with the following specifications:

- Input layer: 561 linear neurons (corresponding to real-valued input features).
- All neurons in the hidden layers use the tanh activation function.
- Hidden layers: Experiment with different numbers of hidden layers and different numbers of neurons in each hidden layer:
  (a) Single hidden layer architectures:
    - Architecture 1: Input layer → 10 neurons → Output layer
    - Architecture 2: Input layer → 100 neurons → Output layer
    - Architecture 3: Input layer → hidden layer (64 neurons) → Output layer
    - Architecture 4: Input layer → hidden layer (512 neurons) → Output layer
  (b) Two hidden layer architectures:
    - Architecture 1: Input layer → hidden layer (64 neurons) → 16 neurons → Output layer
    - Architecture 2: Input layer → hidden layer (128 neurons) → hidden layer (32 neurons) → Output layer
    - Architecture 3: Input layer → hidden layer (256 neurons) → hidden layer (64 neurons) → Output layer
    - Architecture 4: Input layer → hidden layer (512 neurons) → hidden layer (128 neurons) → Output layer
- Output layer: 6 neurons ($Softmax$ activation function).

3. Train the neural network using:
   - Optimizer: Stochastic Gradient Descent (SGD)
   - Loss function: Cross Entropy
4. After completion of the training save the model to a folder. *Hint: take a look at corresponding sample code*
5. Plot a graph for epoch vs loss for training dataset for all architecture.
6. Evaluate the different architectures using validation data and obtain their accuracy.
7. Choose one architecture from single hidden layer and one from two hidden layers(with best validation accuracy).
8. Evaluate the best chosen architecture from single hidden layer and two hidden layer architectures using test data and print following metrics
   - Confusion matrix
   - Accuracy
   - Micro and macro averages of precision, recall, and F1-score.

Part D: Single and Two hidden layer Neural network architecture on the PCA Transformed Dataset (99% Variance)

Perform neural network classification and performance metric evaluation as listed below.

1. Load train, validation, and test data from PCA transformed dataset (99% variance): '`Activity_PCA99_Train.csv`', '`Activity_PCA99_Validation.csv`', '`Activity_PCA99_Test.csv`', respectively.
2. Implement a neural network for classification using PyTorch with the following specifications:
   - Input layer: 156 neurons (corresponding to real-valued input features).
   - All neurons in the hidden layers use the tanh activation function.
   - Hidden layers: Experiment with different numbers of hidden layers and different numbers of neurons in each hidden layer:
     (a) Single hidden layer architectures:
       – Architecture 1: Input layer → hidden layer (156 neurons) → Output layer
       – Architecture 2: Input layer → hidden layer (512 neurons) → Output layer
       – Architecture 3: Input layer → hidden layer (64 neurons) → Output layer
     (b) Two hidden layer architectures:
       – Architecture 1: Input layer → hidden layer (156 neurons) → hidden layer (156 neurons) → Output layer
       – Architecture 2: Input layer → hidden layer (512 neurons) → hidden layer (64 neurons) → Output layer
       – Architecture 3: Input layer → hidden layer (256 neurons) → hidden layer (64 neurons) → Output layer
       – Architecture 4: Input layer → hidden layer (64 neurons) → hidden layer (32 neurons) → Output layer
   - Output layer: 6 neurons ($Softmax$ activation function).
3. Train the neural network using:
   - Optimizer: Stochastic Gradient Descent (SGD)
   - Loss function: Cross Entropy
4. After completion of the training save the model to a folder. *Hint: take a look at corresponding sample code*
5. Plot a graph for epoch vs loss for training dataset for all architecture.

6. Evaluate the different architectures using validation data and obtain their accuracy.
7. Choose one architecture from single hidden layer and one from two hidden layers(with best validation accuracy).
8. Evaluate the best chosen architecture from single hidden layer and two hidden layer architectures using test data and print following metrics
   - Confusion matrix
   - Accuracy
   - Micro and macro averages of precision, recall, and F1-score.

Part E: Comparison and Summarization of Accuracy for each datasets and different architectures on Test Data

In the end, summarize the accuracy in the form of a table. The sample code is also shown below.

```
# Define the table data
data = {
  "Dataset": ["Original", "Scaled", "PCA All", "PCA 99"],
  "Single hidden layer model 1": [100, 100, 100, 100],
  "Single hidden layer model 2": [100, 100, 100, 100],
  "Single hidden layer model 3": [100, 100, 100, 100],
  "Two hidden layer model 1": [100, 100, 100, 100],
  "Two hidden layer model 2": [100, 100, 100, 100],
  "Two hidden layer model 3": [100, 100, 100, 100],
  "Two hidden layer model 4": [100, 100, 100, 100]
}

# Create DataFrame
df = pd.DataFrame(data)
df.set_index("Dataset", inplace=True)

# Print DataFrame
print(f"\nTest Accuracy Comparison\n{df}")
```

Sample NN - PyTorch Code

The following code skeleton (for 2 hidden layer NN) can be copied into a Jupyter notebook. Change the parameters/values as you see fit.

1. Write separate class for single hidden layer NN and two hidden layer NN.

2. Write training and metrics evaluation as separate functions to be reused across different dataset and architectures.

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import LabelEncoder

# Sample for loading tensor data
train_orig = pd.read_csv('data/DateFruit_Train.csv')

X_train_orig = train_orig.drop(columns=['Class'])
y_train_orig = train_orig['Class']

X_train = torch.tensor(X_train_orig.values, dtype=torch.float32)
 ↪  # or torch.long for integer data

# Mapping class labels to int data
le = LabelEncoder()
y_train_encoded = le.fit_transform(y_train_orig)
y_train = torch.tensor(y_train_encoded, dtype=torch.long)

# 1. Create a Neural Network model
class NeuralNet(nn.Module):
  def __init__(self, input_size, h1, h2, output_size):
    super().__init__()
    self.layers = nn.Sequential(
        nn.Linear(input_size, h1), # input layer has linear
         ↪  neurons
        nn.Tanh(), # activation function for hidden layer 1
        nn.Linear(h1, h2),
        nn.Tanh(), # activation function for hidden layer 2
        nn.Linear(h2, output_size)
    )

  def forward(self, x): # forward step applies the model to a
   ↪  given input and computes the various results and
   ↪  gradients
        return self.model(x)

# 2. Initializing model
model = NeuralNet(input_size=30, hidden1=128, hidden2=32,
 ↪  output_size=6)

# 3. Initializing loss function, optimizer
```

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=lr)

# 4. Training (one step)
model.train() # make sure the model is in training mode ie the weights can be updated
outputs = model(X_train) # predict results for the training data
loss = criterion(outputs, y_train) # compute the loss using the cross entropy instance

loss.backward() # back propagation of the loss
optimizer.step() # updating the weights based on the optimizer computed gradients

train_loss = loss.item() # fetch total training loss

# 5. Prediction for validation/test data
model.eval()
with torch.no_grad():
    outputs = model(X)
    pred = torch.argmax(outputs, dim=1)
    loss = criterion(pred, y).item()

# 6. Convergence criteria
if abs(train_loss - prev_train_loss) < threshold:
    counter += 1
else:
    counter = 0
prev_train_loss = train_loss

if counter >= patience:
    print(f"\nConvergence reached at epoch {epoch}")
    break

# Adding the model to a dictionary
two_models[(h1, h2)] = model

# Saving the models to the folder
os.makedirs("saved_models", exist_ok=True) # make sure the directory exists

for (h1, h2) in two_models:
    model = two_models[(h1, h2)] # fetch model from dictionary
```

```
    model_path = f"saved_models/model_34_{h1}_{h2}_7.pth"
    torch.save(model.state_dict(), model_path) # save model to
     ↪  the file in saved_models directory
```

Hint: for your experiments

1. *Choose learning rate* ***'lr'*** *of 0.01 .(values 0.01 ≤* ***lr*** *≤ 0.001 considered good values for learning rate)*
2. ***'patience'*** *is the number of consecutive iteration where there is minimal change in training loss to observe convergence.*
3. *Choose* ***'patience'*** *value to be 1% of the number of iterations/epochs, or, minimum of 10 epochs. (but the norm is about 10% for strong confidence in convergence)*
4. *Choose* ***'threshold'*** *value to be* ***1e-3*** *i.e, change in training loss < 0.001 per iteration for convergence. (<for your understanding>also observe results for* ***5e-4*** *and* ***1e-4****)*

Deliverables

Provide Jupyter notebook containing:

1. Code for all parts (A-E).
2. Plots & Summaries: confusion matrices, and any other visualizations you deem useful.
3. A summary table of metrics for each method.

Zip the notebooks and data into a file named `your.rollnumber_Assignment8.zip` and submit it to Moodle.